



3

9

1080 SP

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Exploring the Essence of the web



د. روان قرعوني

محتوى مجاني غير مخصص للبيع التجاري

3/11/2024

RB Informatics;

لغات البرمجة

■ سنوضح في هذه المحاضرة المفاهيم والتطبيقات الأساسية

Basic concepts and application: HTML, HTTP, URL

بدايةً حدد Tim Berners-Lee المكونات الأساسية للويب وهي تمثل جوهر الويب:

1. HTML

(Hyper Text Markup Language):

هي لغة الترميز الأساسية التي تستخدم لبناء هيكل صفحات الويب وتحديد محتواها أي شكل الصفحة.

2. URL

(Uniform Resource Locator):

هو كعنوان الصفحة والذي من خلاله نستطيع تمييز كل صفحة ويب عن غيرها.
مثال:

<http://www.example.com/index.html>

3. HTTP

(Hyper Text Transfer Protocol):

يسمح لي التواصل مع الصفحة (الموقع) عن طريق http أي أستطيع طلب صفحة web بالإضافة لإرسال واستقبال البيانات من server.



خلاصة:

- لكل صفحة URL يوصلنا لها.
- طلب الصفحة عن طريق بروتوكول http.
- أستطيع تمثيل الصفحة ورسمها عن طريق HTML.

بنية URL

لتمثيل URL لأي صفحة تم الاتفاق على الشكل التالي:

URL: Schema://host[: port]/path[? Query=string][#Anchor]

عند فتح أي Browser يمكن ملاحظة مجموعة من الرموز والمسارات في أعلى الصفحة لنوضح هذه البنية ونشرح جميع أقسامها.

مثال:

http://www.example.com:8090/test/index.html?name=Ahmad&x=true #ourservices

لنوضح هذه الأقسام:

1. Schema:

وهو يعبر عن البروتوكول المستخدم للوصول إلى الانترنت وتبادل المعلومات بين الـ Browser والـ Server وله عدة أنواع مثل: http, https, FTP (File Transfer Protocol)

2. Host:

أي العنوان الفريد الذي يستخدم لتحديد موقع معين على الانترنت، عملياً هو IP (Internet Protocol). حيث نكتب الـ Domain name والـ DNS يقوم بترجمته للـ IP. مثل: www.example.com

3. Port: optional (اختياري)

إذا كان لدي أكثر من مهمة على هذا الـ server، فهو يقوم بتمييز المهمة المطلوبة التي تعمل على الجهاز. ملاحظة: كل port هو unique (فريد من نوعه)، تكون القيمة الافتراضية للـ port هي 80 في http و 443 في https، ويمكن تحديده، مثال: 8090

4. Path:

يقصد به مسار الصفحة التي يريد المستخدم الوصول لها حيث كل صفحة يكون مسارها ضمن مجلدات، مثال: Test/index.html

5. Query String: optional (اختياري)

عندما نقوم بفتح صفحة ويب، قد نرى بارامترات ممررة للصفحة عن طريق الـ URL، تكون مضافة في نهاية الرابط بعد علامة الاستفهام (?)، هذه البارامترات تستخدم لإرسال بيانات من المتصفح إلى الخادم لمعالجتها أو عرض محتوى بناءً على القيم في الـ parameters.

صيغة ال parameters:

تبدأ البارامترات بعلامة استفهام (?), كل بارامتر يتكون من اسم البارامتر وقيمتة يفصل بينهما (=) , أمثلة:
لو بحثنا في Google عن كلمة Test نلاحظ في رأس الصفحة الرابط التالي:

<https://www.google.com/search?q=Test>

القسم الأول <https://www.google.com/search?q=Test>:

هو الذي يحدد عنوان الصفحة المراد زيارتها

بينما القسم الثاني q=Test:

هو الذي سيحدد البارامتر المراد تمريره إلى هذه الصفحة حتى يتم الاستفادة منه حيث نحصل على نتيجة البحث حسب الكلمة الممررة، وهي في هذه الحالة 'Test'
إذا كان هناك أكثر من بارامتر يتم الفصل بينهما باستخدام علامة & مثال: ?id=31&name=Ahmad , هنا: البارامتر الأول id بقيمة 31 والبارامتر الثاني name بقيمة Ahmad.

6. Anchor:

يكون في نهاية URL بعد #

غالباً تكون في التصميم الحديثة للويب (one page design) حيث تكون جميع المعلومات مضمنة في صفحة واحدة بدلاً من التنقل بين الصفحات وتغيير المحتوى بأكمله.
مثال: #ourservices يوجد هذا الزر عند الضغط عليه يقوم بأخذنا إلى نهاية الصفحة ليظهر لنا المعلومات حول الموقع. اختصاراً: عند الدخول إليه لا يفتح صفحة جديدة وإنما يقوم بنقلنا إلى مكان محدد ضمن الصفحة نفسها.

HTTP Protocol

HTTP: هو بروتوكول للتواصل بين client والserver وهذه العملية معتمدة على request والresponse. أي عندما يقوم المستخدم بفتح رابط ما في المتصفح (Browser)، يرسل العميل طلباً (Request) للحصول على الصفحة المطلوبة، ويقوم الخادم (Server) بإرسال استجابة (Response) تحتوي على البيانات المطلوبة لعرض الصفحة.



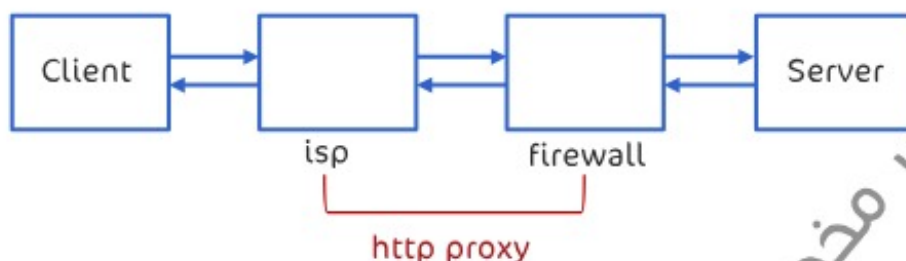
ببساطة: يقوم client بإرسال request إلى server مثلاً صفحة ما فيظهر لي server هذه الصفحة.

ملاحظة هامة:

إذا كانت الصفحة الموقع تحوي فيديوهات أو صور أو روابط أي تحوي Hyper Link عندها قد تضطر لعمل Request جديد، فليس بالضرورة أن تظهر النتائج مع كامل محتويات الصفحة من Request واحد.

هل دائماً ال client يتواصل مع ال server والواجهات مباشرة Direct؟

يستحيل ذلك، بل يتم التواصل عن طريق ال http proxy أيضاً، تتم معالجة ال Request ثم إرسالها لل server. قد يوجد أكثر من http proxy قبل الوصول للوجهة المطلوبة (يختلف هذا العدد بين الشركات).



فائدة:

http proxy: هو وسيلة تستخدم لتوجيه طلبات من ال client مثل متصفح النت إلى ال server أي أنه حلقة بين ال client وال server ويقوم بعدة وظائف وأهمها الأمان بحيث لن يعرف ال IP Address للمستخدم بشكل مباشر ولكن يعرف ال IP Address لل server أو الحاسوب.
(Internet Server Provider) : isp

عملية ال Request & Response

هي عملية أساسية تستخدمها الأجهزة للتواصل مع بعضها البعض في الشبكة مثلاً يرسل ال client طلباً ببعض ال Data ويستجيب ال server لهذا الطلب، تحديداً هو نمط تبادل من خلاله يُرسل ال client طلباً Request إلى ال server للحصول على مجموعة من البيانات والذي بدوره يتلقى الطلب ويعالجه ويعيد في النهاية رداً Response.

بنية ال Request:

الطلب الذي يرسله ال client ليحصل على Response تكون بنيته على الشكل التالي:

Method/path http/version

Header 1: value

Header 2: value

....

[Request Body] optional



لنوضح هذه المكونات:

1. Method: عندما نقوم بإرسال ال Request نرسل أولاً ال Method والتي من أهم أنواعها (Get, post, ..), سنقوم بشرحها لاحقاً.
2. Path: هو المسار المطلوب تطبيق ال Method عليه.
3. http/version: هو رقم إصدار البروتوكول http المستخدم.
4. Header: تمثل باراترات ممررة إلى server, معلومات إضافية ترسل مع الطلب مثل نوع المحتوى أو معلومات المصادقة.
5. Body: يحتوي على البيانات المرسله من ال client إلى server (يستخدم عادة مع post أو put), قد يتضمن بيانات نموذج.

⚡ **بنية ال Response:**

http/version status code exmplation

Header 1: value

Header 2: value

..

..

[Response Body] optional

لنوضح هذه البنية:

1. http version: هو البروتوكول المستخدم ورقم إصداره.
2. Status code: هو الرقم الي يدل على حالة ال Response, يظهره لنا ال server, لمعرفة ما إذا كان الطلب نجح أم فشل, وسيتم شرحه لاحقاً.
3. Header name: يحتوي على معلومات إضافية حول ال Response مثل:
Content_Type: text/html
Content_Length: 123
4. Response Body: البيانات المعادة يمكن أن يحتوي على code (HTML, CSS, JS).

- Postman: هي tool تستخدم في اختبار APIs أي اختبار واجهات برمجة التطبيقات.
- Postman collection: أداة قوية لتنظيم وإدارة الطلبات الخاصة بواجهات برمجة التطبيقات, مما يسهل عملية التطوير والاختبار والتعاون بين الفريق. ستكون واضحة بالعمل مع الفريق لاختبار ال APIs.



نصائح عملية:

لتسهيل العمل بين مطورين backend ومطورين front-end فيجب الاتفاق مسبقاً على شكل الـ response التي سترجع للـ user ، مثلاً عند العمل على تطبيق ما يتطلب تسجيل الدخول وإدخال معلومات كالاسم وتاريخ الميلاد

هنا يمكن كتابة تاريخ الميلاد بعدة طرق، أي:

- يوم/شهر/سنة
- شهر/يوم/سنة

وبهذه الحالة يجب الاتفاق على شكل الـ response ليكون ثابت في كافة الإدخالات وبعدها يتم الربط بين كلا القسمين

نتيجة: الـ Postman يساعدنا في اختيار الـ Header أي يعرض لنا خيارات على حسب الطلب.

لنوضح الـ Request والـ Response بوضع أمثلة:

Request:

Get/test/index.html?name=Ahmad http 1.1

Host: www.example.com

Get: هي الـ method المستخدمة، بعدها يأتي الـ path وليس لها body.

Host: www.example.com ← Header يمثل الموقع وهو عبارة عن name: value

هنا لدينا Header واحد ولكن قد يكون هناك أكثر من Header.

- Get ليس لها body إذاً لا يوجد Request body.
- والآن نكون قد أرسلنا الـ Request من الـ client إلى server وبالتالي سيظهر لنا الـ Response كما يلي للـ client.

Response:

http/1.1 200

content_type: text/html

content_length: 850

<html>

<h>

<title> </title>

</h>

<body>

</body>

</html>

- http/1.1: البروتوكول http إصدار 1.1

- State code=200 يدل على نجاح العملية

- Content type و Content length: بارامترات

Headers إضافية مساعدة، تدل على نوع المحتوى

(html) وعدد محارفه: 850

فائدة:

قد يكون لدينا Request داخل Request آخر، مثلاً في حال طلب صورة إلى جانب html تكون الصورة غير واضحة لعدد من الثواني ثم بعد قليل تظهر بشكل واضح والسبب هو أن الـ Response القادمة قامت بإعادة طلب Request آخر.

Method type

إن من أشهر أنواع الـ methods:

Get: تستخدم لأخذ معلومات من الـ Server لكنها لا تحوي على body في بنيتها حيث نمرر المعلومات التي نطلبها ضمن الـ URL

Post: تستخدم لإرسال أو تعديل المعلومات

Post/test/index.html HTTP/1.1
Host: www.example.com
Request body ←

Name = ahmad
X = true



الفرق بين الـ Post و الـ Get:

الأمان:

تعتبر الـ Post أعلى حماية وأمان من الـ Get وذلك لأن في الـ Get ترسل بيانات الطلب كجزء من عنوان URL مثال: `http://example.com/page?name=Ahmad` وهذه البيانات تظهر بشكل واضح في شريط العنوان، وبالتالي يمكن رؤيتها بسهولة سواء في سجلات السيرفر أو في تاريخ التصفح أو عند مشاركة الرابط مع الآخرين، فهي غير مناسبة للبيانات الحساسة كالبيانات الشخصية وكلمات المرور. أما في الـ Post تُرسل البيانات في الـ Request Body فلا تكون مرئية في الـ URL لذلك تعتبر أكثر أماناً عند التعامل مع بيانات حساسة لأنها لا تُعرض في شريط العنوان أو سجلات السيرفر بشكل مباشر. ومع ذلك لا تعتبر الـ Post أمنة بشكل كامل.

المحدودية:

الـ URL للـ GET له محدودية حيث لا يمكن تمرير باراميترات كثيرة ضمنه وهذا ما يجعلها غير مناسبة لنقل كميات كبيرة من البيانات، أما بالـ Post يمكن تمرير عدد كبير من الباراميترات حيث لا يوجد قيود على طول البيانات التي يتم إرسالها في الـ (Request Body)، وهذا ما يجعل الـ Post ملائمة أكثر لإرسال البيانات والملفات.



Head: هو الـ Get عمليا ولكنها تعيد الـ response من دون body أي تعيد الـ header فقط تتميز الـ head بأنها تكون أسرع في التحميل من الـ get لأنه عند حصول ضغط على الشبكة فإن الحزمة bandwidth تصبح بطيئة لكن الـ head لا يقوم بإرجاع response content بالتالي فهي أسرع. سلبياها: إذا كانت الصفحة غير محفوظة أو تم حفظها قديما أو تتجدد باستمرار نكون قد خسرنا طلب الـ request من دون فائدة لذلك سوف نحتاج إلى إرسال الـ get مرة أخرى.

سؤال: إذا كان الـ head يعيد الـ header فقط ما الفائدة من وجوده؟

في المتصفحات الحديثة يتم حفظ اخر الصفحات التي تم زيارتها والدخول اليها في الـ cache وذلك لتوفير الوقت وإن أكبر استخدام الـ head هو عند زيارة صفحة قد تم زيارتها من قبل ومازالت محفوظة في الـ cache فلا يقوم بإعادة تحميلها، يقوم الـ Browser بإرسال الـ header للـ server يطلب فيه تريخ آخر تعديل للصفحة (last modified) وعندما يقارنه مع تاريخ تحميل الصفحة فإذا كان أقدم منه فلا يقوم بإعادة تحميله لأنه يمتلك الصفحة باخر تحديثاتها وإذا كان الـ last modified أحدث يقوم بإعادة تحميلها من جديد وهنا لا حاجة لاستخدام الـ get لأننا بحاجة معلومات بسيطة جدا في الـ response.

نتيجة:

Head	Get
تعيد response فقط أي بدون body	تعيد response مع response body

(Put, Delete): شبيهين بالـ Post كبنية Structure

Put: إضافة معلومات بالـ server

Delete: هي method خاصة بالحذف

Status code

هو عبارة عن رقم يقوم الـ server بإرساله يكون موجود في الـ response ليعطيني نتيجة الـ Requeste.

يتكون من خمس حالات:

1. Status code التي تبدأ بالعدد 1: 100 -> 199

أي كل Status code تبدأ بالعدد 1 تكون عبارة عن information أي تقوم بتقديم معلومات مثال: (continue) 100 (نادرا ما تستخدم): المقصود من هذا الرقم ان جزء من الـ request وصل إلى وجهته لذلك قم بإرسال باقي الأجزاء

2. Status code التي تبدأ بالعدد 2: 200 -> 299

تكون عبارة عن successful response أي نجح الطلب.



مثال 1: (ok) 200 تدل على أن الطلب الذي أرسلناه قد تم بنجاح.

مثال 2: (created) 201 مثلاً عند طلب إنشاء ملف جديد على الـ server يصلنا رد 201 أي يتم إنشاء الملف.

3. Status code التي تبدأ بالعدد 3: 300 -> 399

هي عبارة عن additional action أي يطلب من الـ user ان يرسل action إضافي الى جانب الـ request المرسل
مثال: 301 يقصد به أن الـ resource الذي طلبته ما زال موجوداً لكنه انتقل الى مكان آخر أي تغير مساره فتكون الـ response في هذه الحالة المسار الجديد الذي سنعيد طلبه

4. Status code التي تبدأ بالعدد 4: 400 -> 499

هي عبارة عن request error، أمثلة:

400 (bad request): ويظهر في حال تم ارسال request خاطئ من قبل الـ client او الـ front

مثال: request ما يقوم بأخذ id من النوع integer لكن عن طريق الخطأ قمت بإرسال rd=2 فعندها سيكون الـ request خاطئ

401 (unauthorized): يقصد به أن هذا الـ request الذي قمت بطلبه أنت غير مخول بطلبه أي أن المستخدم لا يملك صلاحية الدخول لهذه الصفحة

403 (forbidden): يظهر إذا كان الطلب الذي تقوم بطلبه ممنوع كلياً وغير مسموح به

404 (not found): يقوم المتصفح بإظهار الصفحة 404 عندما يقوم المستخدم بطلب صفحة غير موجودة

5. Status code التي تبدأ بالعدد 5: 500 -> 599

هي عبارة عن server error، أمثلة:

500 (internal server): يظهر إذا وجد خطأ exception في الكود الخاص بالباك أي اذا قام شخص بطلب services صحيحة و request صحيح وهو authorized فعند وصوله الـ server ودخوله الـ function الذي سيقوم بتنفيذ الكود يظهر exception في الباك عندها يظهر انه حدث خطأ من ناحية الخادم.

503 (service unavailable): قد يكون الموقع المطلوب في حالة صيانة فيظهر 503 أي ان الموقع موجود لكنه غير متاح حالياً.

وللحلم بقية..!

See You Soon

